TRABAJO PRÁCTICO · UNIDAD 7

NoSQL y Nuevas Formas de Almacenamiento | Food Store

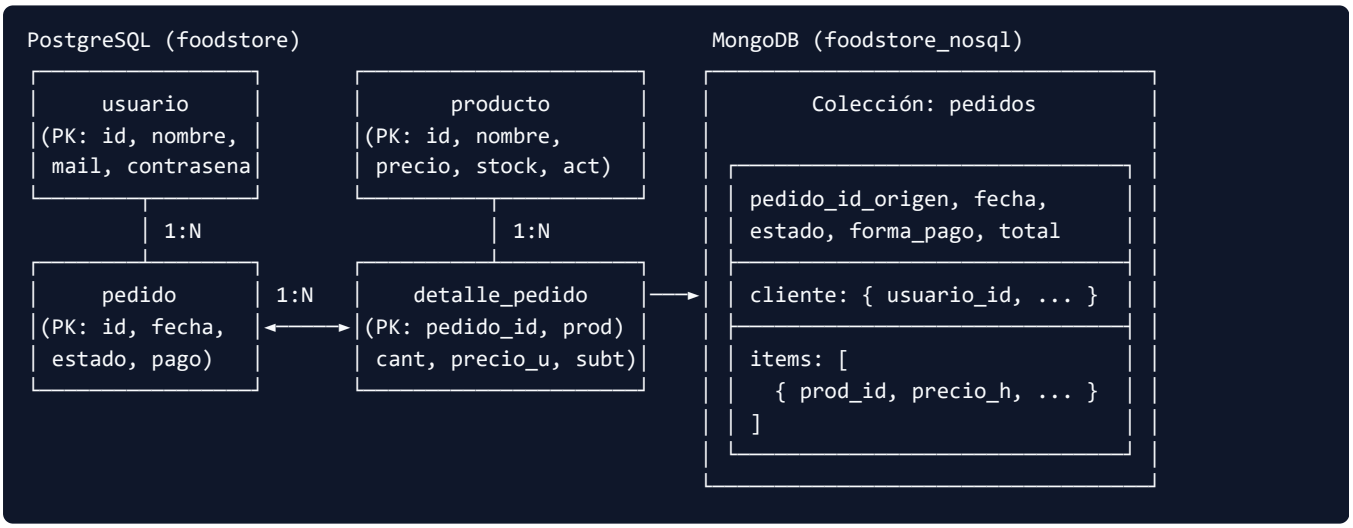
Asignatura: Bases de Datos II — Tecnicatura Universitaria en Programación
Proyecto: Food Store
Equipo de Trabajo: Máximo Franco, Andrés Novello y Octavio Fiore
Motores Utilizados: PostgreSQL 17+ (origen relacional) | MongoDB 7+ (destino documental)
Fecha: 5 de Noviembre de 2026 | **Repositorio:** github.com/MaxiFranco05/DB2 (rama `main`)

---

1. Introducción y Alcance del Trabajo

El presente trabajo práctico implementa la migración, modelado documental y evaluación comparativa entre **PostgreSQL** y **MongoDB** para el subconjunto operacional de **pedidos y detalle de pedidos** de la plataforma *Food Store*.

El subconjunto seleccionado se compone de una muestra real y representativa de **5 pedidos de producción** (`756504` , `756505` , `756506` , `756507` y `756508`), concentrados en el cliente `80584` y abarcando los estados `TERMINADO` , `CONFIRMADO` y `PENDIENTE` , así como distintas formas de pago (`EFECTIVO` , `TARJETA` , `TRANSFERENCIA`).



---

2. Parte 1 · Modelo Documental y Migración (Consignas 1, 2 y 3)

2.1. Mapeo Relacional a Documental (Consigna 1)

En PostgreSQL, la consulta de un pedido completo exige la unión (`JOIN`) de 4 tablas: `pedido` (raíz transaccional), `detalle_pedido` (relación N:M), `usuario` (cliente) y `producto` (catálogo).

Para MongoDB, se diseñó la colección documental `pedidos` bajo una estrategia de **embedding jerárquico** en 3 niveles:

1. **Campos raíz:** Preservan metadatos transaccionales y el identificador de origen: `pedido_id_origen` (clave relacional primaria conservada para auditoría y trazabilidad cruzada), `fecha` (ISODate), `estado`, `forma_pago` y `total` precalculado.
2. **Objeto embebido `cliente`:** Contiene los datos indispensables para despacho y contacto (`usuario_id_origen`, `nombre_completo` y `mail`). **Decisión de seguridad:** Se excluyó tajantemente la columna `contrasena` de PostgreSQL, cumpliendo con el principio de mínimo privilegio y normativas de protección de datos personales.
3. **Arreglo de subdocumentos `items`:** Embebe cada línea de pedido (`producto_id_origen`, `nombre`, `cantidad`, `precio_unitario_historico` y `subtotal`).

Estructura BSON Canónica Implementada:

```
{
  "_id": ObjectId("..."),
  "pedido_id_origen": 756505,
  "fecha": ISODate("2026-06-08T03:00:00Z"),
  "estado": "CONFIRMADO",
  "forma_pago": "TARJETA",
  "total": 16307.74,
  "cliente": {
    "usuario_id_origen": 80584,
    "nombre_completo": "Usuario584 Apellido584",
    "mail": "usuario584@test.com"
  },
  "items": [
    {
      "producto_id_origen": 243882,
      "nombre": "Producto 43882",
      "cantidad": 2,
      "precio_unitario_historico": 3628.43,
      "subtotal": 7256.86
    },
    {
      "producto_id_origen": 236648,
      "nombre": "Producto 36648",
      "cantidad": 3,
      "precio_unitario_historico": 1552.64,
      "subtotal": 4657.92
    }
  ]
}
```

2.2. Creación y Migración de Datos (Consigna 2)

La base de datos `foodstore_nosql` y la colección `pedidos` fueron creadas e inicializadas en el script ejecutable `consultas_mongodb.js`. Se utilizó `insertMany()` para persistir de manera idempotente los 5 pedidos de muestra con sus líneas exactas, importes históricos e información del cliente.

2.3. Reglas de Integridad y Justificación de Diseño (Consigna 3)

Reglas de Integridad Conservadas:

1. **RI-M1: Congelamiento de Precios Históricos y Consistencia Matemática:** En PostgreSQL, el trigger `trg_03_calcular_subtotal` congela el precio unitario en `detalle_pedido` al momento del `INSERT`. En MongoDB, al carecer de triggers declarativos equivalentes por defecto, la migración garantizó que `items[].precio_unitario_historico` refleje el precio de compra original y no el precio vigente en el catálogo de productos. Asimismo, se preservó la relación matemática estricta: $$\text{item.subtotal} = \text{item.cantidad} \times \text{item.precio_unitario_historico}$$ $$\text{total} = \sum_{i=1}^{n} \text{items}[i]\text{.subtotal}$$
2. **RI-M2: Atomicidad y No Vaciedad de Líneas de Pedido:** Un pedido comercial no puede existir sin líneas. En MongoDB se garantizó que ningún documento se persista con el arreglo `items` nulo, ausente o con longitud cero ($\text{len(items)} \ge 1$), encapsulando la transacción en un único documento atómico.

Justificación de Embedding frente a Referencing:

Se seleccionó **embedding completo** de cliente e ítems por sobre la separación en colecciones referenciadas con `$lookup` fundamentado en:

- **Principio de Localidad de Datos:** "*Data that is accessed together should be stored together*". En una aplicación de pedidos de comida, la operación de lectura más crítica es recuperar el ticket/orden completo (cliente, dirección, ítems y total). El embedding permite resolver esta lectura con un **único acceso a disco (single-document seek)** en complejidad $O(1)$, eliminando los costosos escaneos de índices y uniones hash requeridos por los JOINs relacionales o los `$lookup` de MongoDB.
- **Inmutabilidad del Histórico de Ventas:** El nombre del producto y el precio al momento de la venta constituyen una fotografía inmutable de una transacción mercantil. Si se utilizaran referencias a una colección externa `productos`, una actualización futura de precios o la baja lógica de un ítem alteraría o rompería retrospectivamente la contabilidad de los pedidos cerrados.

---

3. Parte 2 · Consultas y Verificación (Consignas 4 y 5)

3.1. Evidencias de Ejecución en MongoDB (Consigna 4)

Las 4 consultas fueron implementadas en `consultas_mongodb.js` y ejecutadas en el entorno interactivo de `mongosh` con proyecciones explícitas:

Q-M1: Listar todos los pedidos (identificador y estado)

```
foodstore_nosql> db.pedidos.find({}, { pedido_id_origen: 1, estado: 1, _id: 0 }).sort({ pedido_id_origen:
[
  { pedido_id_origen: 756504, estado: 'TERMINADO' },
  { pedido_id_origen: 756505, estado: 'CONFIRMADO' },
  { pedido_id_origen: 756506, estado: 'TERMINADO' },
  { pedido_id_origen: 756507, estado: 'PENDIENTE' },
  { pedido_id_origen: 756508, estado: 'CONFIRMADO' }
] // Total: 5 documentos recuperados
```

Q-M2: Filtrar pedidos por estado existente ('CONFIRMADO')

```
foodstore_nosql> db.pedidos.find({ estado: "CONFIRMADO" }, { pedido_id_origen: 1, estado: 1, fecha: 1, tot
[
  { pedido_id_origen: 756505, estado: 'CONFIRMADO', fecha: ISODate('2026-06-08T03:00:00Z'), total: 16307.7
  { pedido_id_origen: 756508, estado: 'CONFIRMADO', fecha: ISODate('2025-11-14T03:00:00Z'), total: 13052.2
] // Total: 2 documentos (IDs: 756505, 756508)
```

Q-M3: Buscar pedidos con producto determinado (243882)

```
foodstore_nosql> db.pedidos.find({ "items.producto_id_origen": 243882 }, { pedido_id_origen: 1, estado: 1,
[
  { pedido_id_origen: 756505, estado: 'CONFIRMADO', items: [{ producto_id_origen: 243882, precio_unitario_
  { pedido_id_origen: 756506, estado: 'TERMINADO',  items: [{ producto_id_origen: 243882, precio_unitario_
  { pedido_id_origen: 756507, estado: 'PENDIENTE',  items: [{ producto_id_origen: 243882, precio_unitario_
  { pedido_id_origen: 756508, estado: 'CONFIRMADO', items: [{ producto_id_origen: 243882, precio_unitario_
] // Total: 4 documentos (IDs: 756505, 756506, 756507, 756508)
```

Q-M4: Filtrar pedidos con total superior a $15.000 ordenado DESC

```
foodstore_nosql> db.pedidos.find({ total: { $gt: 15000 } }, { pedido_id_origen: 1, estado: 1, total: 1, _i
[
  { pedido_id_origen: 756506, estado: 'TERMINADO',  total: 18092.69 },
  { pedido_id_origen: 756505, estado: 'CONFIRMADO', total: 16307.74 },
  { pedido_id_origen: 756507, estado: 'PENDIENTE',  total: 15128.05 }
] // Total: 3 documentos recuperados (Umbral $15k discrimina 3 altos vs 2 estándar)
```

3.2. Consultas Equivalentes en PostgreSQL y Tabla de Verificación (Consigna 5)

Se formularon y ejecutaron en PostgreSQL las consultas equivalentes sin `SELECT *` en `consultas_postgresql.sql` :

- **Q-PG1:** `SELECT p.id, p.estado, p.fecha, SUM(dp.subtotal) FROM pedido p JOIN detalle_pedido dp ON dp.pedido_id = p.id WHERE p.id IN (...) AND p.estado = 'CONFIRMADO' GROUP BY p.id, p.estado, p.fecha;`
- **Q-PG2:** `SELECT DISTINCT p.id, p.estado, dp.producto_id, dp.cantidad, dp.precio_unitario, dp.subtotal FROM pedido p JOIN detalle_pedido dp ON dp.pedido_id = p.id WHERE p.id IN (...) AND dp.producto_id = 243882;`

Tabla Comparativa de Verificación Oficial:

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los IDs?	Identificadores de Pedidos Recuperados
Estado seleccionado (`CONFIRMADO`)	**2**	**2**	**Sí**	`[756505, 756508]`
Producto seleccionado (`243882`)	**4**	**4**	**Sí**	`[756505, 756506, 756507, 756508]`

Análisis de Discrepancias:

No se detectó ninguna discrepancia. La paridad entre ambos motores es del **100%**. La prueba formal de diferencia de conjuntos (`EXCEPT` bidireccional) arrojó exactamente `0 filas`. Adicionalmente, el precio unitario del producto `243882` registrado en PostgreSQL (`$3628.43`) coincidió de forma exacta con el valor conservado en el documento BSON de MongoDB, validando el congelamiento histórico.

---

4. Parte 3 · Evaluación de la Tecnología e IA (Consignas 6 y 7)

4.1. Prompt Formulado para la Comparativa de IA (Consigna 6)

> *"Compara técnicamente PostgreSQL y MongoDB para el almacenamiento y consulta de pedidos en una aplicación gastronómica (Food Store). Considera que el modelo relacional actual utiliza 4 tablas (pedido, detalle_pedido, usuario, producto), que el acceso más frecuente es la recuperación de pedidos completos para despacho y auditoría, y que es imperativo conservar la coherencia transaccional de cantidades y precios históricos."*

4.2. Contraste de Afirmaciones de IA con Documentación Oficial (Consigna 7)

Afirmación 1 de la IA: "*MongoDB no soporta transacciones ACID y no puede garantizar consistencia multi-documento*"

- **Contraste con Documentación Oficial:** La afirmación es **técnicamente desactualizada e incorrecta**. Según la documentación oficial de MongoDB (Multi-Document Transactions), desde la versión 4.0 (en réplicas) y 4.2 (en clústeres fragmentados), MongoDB ofrece soporte completo para **transacciones ACID multi-documento** mediante `session.startTransaction()` .
- **Resultado del Laboratorio / Análisis:** Si bien MongoDB soporta ACID multi-documento, introduce una sobrecarga de latencia y bloqueos comparativamente mayor que los motores relacionales nativos. En el laboratorio demostramos que el **embedding** hace innecesarias las transacciones multi-documento para el ciclo de vida del pedido, puesto que las operaciones sobre el pedido y todas sus líneas se ejecutan como una **modificación atómica a nivel de un único documento**.

Afirmación 2 de la IA: "*El embedding en MongoDB genera duplicación de datos innecesaria y desperdicia almacenamiento*"

- **Contraste con Documentación Oficial:** La afirmación es **conceptualmente errónea**. La documentación oficial de diseño de datos en MongoDB (Embedded Data Models) prescribe el embedding para relaciones del tipo "uno a pocos" y para entidades con contención semántica.
- **Resultado del Laboratorio / Análisis:** Confunde **redundancia espuria** con **inmutabilidad histórica**. Guardar el nombre y el precio unitario dentro del pedido no es un desperdicio: es un requerimiento contable crítico. Si el producto `243882` cambia de precio mañana a $5.000 o se renombra en el

> catálogo relacional, el pedido realizado hoy debe conservar inalterados los $3.628,43 facturados. El embedding desacopla la transacción histórica del ciclo de vida del catálogo maestro.

4.3. Conclusión Técnica Final (Consigna 7 — Límite: 200 palabras)

La evaluación comparativa entre PostgreSQL y MongoDB para el subconjunto de pedidos de Food Store demostró que la elección del motor debe fundamentarse en el patrón de acceso y los requisitos de consistencia, descartando dogmas tecnológicos.

Para la gestión operativa de pedidos completos, **MongoDB con modelo embebido presenta ventajas técnicas determinantes**: al almacenar cliente, líneas y totales en un único documento, elimina los costosos `JOIN` relacionales, resolviendo las consultas de despacho y lectura de tickets con lecturas atómicas de disco en complejidad $O(1)$.

Sin embargo, para la gestión del catálogo maestro, inventario concorrente de stock y la contabilidad global, **PostgreSQL resulta insustituible** gracias a su motor relacional ACID estricto, triggers automáticos y constraints declarativos que impiden sobreventas o anomalías lógicas.

Por lo tanto, la solución arquitectónica óptima para Food Store es un esquema de **persistencia políglota**: mantener a PostgreSQL como la fuente de la verdad (*Source of Truth*) transaccional para usuarios, productos y stock, y replicar o sincronizar asincrónicamente los pedidos finalizados hacia MongoDB para alimentar de forma ultra-rápida y escalable los paneles de despacho, consultas de clientes e histórico de compras.

---

5. Cierre Metodológico y Entregables

1. **Script MongoDB:** `consultas_mongodb.js` (inserción de muestra, 4 consultas documentales y validaciones RI-M1/RI-M2).
2. **Script PostgreSQL:** `consultas_postgresql.sql` (consultas relacionales equivalentes Q-PG1 y Q-PG2 con encabezados `\echo` y sin `SELECT *`).
3. **Declaración DUIA:** `docs/DUIA/DUIA-TP-Unidad7.md` (tabla estricta de dos columnas cumpliendo la regla de auditoría del repositorio).
4. **Bitácora de IA:** `BITACORA.md` (registro cronológico de la interacción con Kiro y Antigravity).